МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка

ЗВІТ

до лабораторної роботи №9
з дисципліни «Професійний Python»

Тема: «Паралельність, багатопроцесність та оптимізація»

Виконав:
студент Левандівський Назар

Перевірив:
Куньо Іван Михайлович

Львів – 2026

Мета роботи

Дослідити вплив GIL на CPU-bound та I/O-bound задачі, застосувати threading, multiprocessing, ThreadPoolExecutor, ProcessPoolExecutor, NumPy і caching, знайти bottleneck за допомогою timeit, cProfile і tracemalloc та експериментально визначити speedup.

1. Постановка задачі та проєкт

Варіант 1. Система обліку студентів. Проєкт `student_manager` з ЛР1 зберігає ім'я, прізвище, групу й середній бал студента; базові можливості – додавання й перегляд, пошук, пошук за групою, найкращий студент, середній бал групи. Потрібно профілювати набір не менше 100 000 студентів (пошук за групами, середній бал усіх і кожної групи, minimum, maximum, standard deviation, рейтинг), порівняти Python loops, NumPy, ThreadPoolExecutor і ProcessPoolExecutor, для I/O-bound частини паралельно завантажувати файли студентів і застосувати caching із invalidation.

Таблиця 1 – Модулі проєкту (src/student_manager)

Модуль	**Призначення**
`models.py, repositories.py`	Student (ЛР1) і сховище у пам'яті
`services.py`	функції ЛР1 та `StudentService` із кешем статистики груп
`analytics.py`	baseline на Python: статистика, групи, рейтинг, індекси
`vectorized.py`	NumPy-версії на колонках масивів
`parallel.py, shared.py`	`ThreadPoolExecutor`, `ProcessPoolExecutor`, `Thread`, `Process`, `Lock`
`loaders.py`	I/O: запис і паралельне завантаження файлів
`profiling.py`	cProfile, tracemalloc, timeit, perf_counter

Test dataset. `generate_students(count, seed=42)` створює студентів із 12 груп (PI, KN, SE, IS × 21–23), 20 імен і 20 прізвищ, бал – рівномірно від 35 до 100 із двома знаками. Розміри: 10 000, 100 000 і 400 000 студентів (для I/O – 8 CSV-файлів по 12 500). Комп'ютер: Intel(R) Core(TM) i7-14650HX, 24 логічних ядер, Windows-11, Python 3.12.10, NumPy 2.5.4. Кожен вимір – 5 повторів, повтори методів чергуються; перед вимірюванням результат кожної реалізації порівнюється з baseline.

2. Baseline і вимірювання часу

Baseline – чисті Python-цикли над списком `list[Student]`: `moments_python` (два проходи: сума/min/max, потім відхилення), групи, рейтинг і `student_score` – CPU-bound рейтинговий індекс з 100 ітерацій sin/cos/sqrt на студента

(формула має аналітичне значення `sqrt(grade² + 100)`, тому правильність перевіряється й незалежно від NumPy).

```
def moments_python(values: Sequence[float]) -> Moments:
    """Два проходи Python-циклами: сума, мінімум, максимум, відхилення."""
    count = len(values)
    if not count:
        return EMPTY_MOMENTS
    total = 0.0
    minimum = maximum = values[0]
    for value in values:
        total += value
        if value < minimum:
            minimum = value
        elif value > maximum:
            maximum = value
    mean = total / count
    squares = 0.0
    for value in values:
        deviation = value - mean
        squares += deviation * deviation
    return Moments(count, mean, squares, minimum, maximum)

def student_score(grade: float) -> float:
    """Рейтинговий індекс: CPU-bound обчислення для одного студента.

    Кожна ітерація дає sqrt(v^2 + sin^2(v) + cos^2(v)) = sqrt(v^2 + 1),
    тому очікуване значення дорівнює sqrt(grade^2 + SCORE_ITERATIONS).
    """
    value = grade
    for _ in range(SCORE_ITERATIONS):
        value = math.sqrt(
            value * value + math.sin(value) ** 2 + math.cos(value) ** 2
        )
    return value
```

Час вимірюється через `time.perf_counter` (повтори методів чергуються), для мікрооперацій кешу – `timeit.repeat`. Baseline на 100 000 студентів: індекси 4,51 s, статистика, групи, рейтинг і пошук – лише мілісекунди, тому саме індекси є кандидатом у bottleneck. Базові можливості ЛР1 збережено (вивід збігається з прикладом методички):

```
PS C:\Github\prof_python\Lab_09> student-manager
Students:
Anna Koval               PI-21      91.50
Oleh Melnyk              PI-21      84.00
Iryna Bondar             PI-22      95.20
Andrii Shevchenko        PI-21      88.70

Students of group PI-21:
Anna Koval               PI-21      91.50
Oleh Melnyk              PI-21      84.00
Andrii Shevchenko        PI-21      88.70

Group average: 88.07
Best student: Iryna Bondar (95.20)
Student found: Oleh Melnyk PI-21
```

3. cProfile і tracemalloc

```
PS C:\Github\prof_python\Lab_09> student-manager profile
Студентів: 100000; student_report під cProfile: 10.41 s
30300082 function calls in 10.409 seconds

   Ordered by: cumulative time
   List reduced from 23 to 15 due to restriction <15>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.001    0.001   10.409   10.409 analytics.py:273(student_report)
        1    0.000    0.000   10.282   10.282 analytics.py:257(calculate_scores_sequential)
        1    0.039    0.039   10.282   10.282 analytics.py:252(scores_chunk)
   100000    6.109    0.000   10.243    0.000 analytics.py:233(student_score)
 10000000    1.451    0.000    1.451    0.000 {built-in method math.cos}
 10000000    1.408    0.000    1.408    0.000 {built-in method math.sin}
 10000013    1.276    0.000    1.276    0.000 {built-in method math.sqrt}
        1    0.001    0.001    0.093    0.093 analytics.py:212(group_statistics_python)
        1    0.039    0.039    0.091    0.091 analytics.py:191(group_moments_python)
       13    0.028    0.002    0.028    0.002 analytics.py:160(moments_python)
   100000    0.020    0.000    0.020    0.000 {method 'setdefault' of 'dict' objects}
        1    0.000    0.000    0.017    0.017 analytics.py:227(ranking_python)
   100000    0.016    0.000    0.016    0.000 {method 'append' of 'list' objects}
        2    0.015    0.008    0.015    0.008 {built-in method builtins.sorted}
        1    0.000    0.000    0.014    0.014 analytics.py:186(statistics_python)
```

Bottleneck – `student_score`. 100 000 викликів, tottime 6,11 s, cumtime 10,24 s із 10,41 s усього – 98,4 %. Саме в цій функції 10 000 000 викликів кожної з `math.sin`, `math.cos`, `math.sqrt` (разом із власним часом функції – 10,24 s). Групи займають 0,09 s, рейтинг – 0,02 s. Задача CPU-bound: час іде на обчислення Python bytecode, без очікування I/O, тому її виконання впирається в GIL. Профайлер уповільнює код: без нього той самий звіт тривав 4,73 s проти 13,99 s під cProfile.

```
PS C:\Github\prof_python\Lab_09> student-manager memory
Операція                        залишилось, MB   пік, MB
generate_students(100000)                13.54     13.55
to_columns (масиви NumPy)                 0.95      0.96
statistics_python                         0.00      0.76
statistics_numpy                          0.00      0.76
```

Пам'ять (tracemalloc). 100 000 об'єктів `Student` займають 13,5 MB, колонки NumPy (бали float64 + коди груп int16) – 0,95 MB, тобто в 14 разів менше. Пік рейтингу: Python 6,1 MB, NumPy 1,9 MB; статистики майже не виділяють пам'яті (за пік відповідає тимчасовий масив 0,76 MB).

4. Потоки: ThreadPoolExecutor і synchronization

```python
def scores_threads(grades: Sequence[float], workers: int) -> list[float]:
    """Рейтингові індекси: кожен потік рахує свій шматок (спільний GIL)."""
    with ThreadPoolExecutor(max_workers=workers) as executor:
        parts = executor.map(scores_chunk, split(grades, workers))
        return [score for part in parts for score in part]

def split(values: Sequence[T], parts: int) -> list[Sequence[T]]:
    """Ділить послідовність на не більше ніж parts шматків."""
    size = max(1, -(-len(values) // parts))
    starts = range(0, len(values), size)
    return [values[start:][:size] for start in starts]
```

`scores_threads` ділить список балів на шматки, і кожен потік пулу рахує свій шматок. Потоки виконують Python bytecode по черзі через GIL, тому для CPU-bound функції очікується відсутність прискорення (результати – у розділі 6).

Synchronization. GIL звільняється лише на блокуючих викликах, а перемикання потоків може статися між будь-якими двома кроками, тому

спільний змінюваний стан треба захищати. `MomentsAccumulator` зливає внески потоків під `Lock`; `GroupTally.register` – «прочитай, обчисли, запиши» без захисту й із ним.

```
class MomentsAccumulator:
    """Спільний накопичувач moments: оновлення захищає Lock."""

    def __init__(self) -> None:
        """Створює порожній накопичувач і Lock."""
        self._moments = EMPTY_MOMENTS
        self._lock = threading.Lock()

    @property
    def moments(self) -> Moments:
        """Накопичені moments."""
        return self._moments

    def add(self, moments: Moments) -> None:
        """Зливає внесок одного потоку; критична секція мінімальна."""
        with self._lock:
            self._moments = self._moments.merge(moments)

class GroupTally:
    """Кількість студентів і сума балів групи, спільні для потоків."""

    def __init__(self, *, synchronized: bool) -> None:
        """Створює лічильники; synchronized вмикає Lock."""
        self.count = 0
        self.total = 0.0
        self._lock = threading.Lock() if synchronized else None

    def register(self, student: Student) -> None:
        """Додає студента до лічильників (критична секція)."""
        with self._lock or nullcontext():
            count = self.count
            total = self.total
            weight = self.weight_of(student)
            self.count = count + 1
            self.total = total + weight

    @staticmethod
    def weight_of(student: Student) -> float:
        """Внесок студента; виклик функції - точка перемикання потоків.

        У реальному коді між читанням і записом майже завжди є такі
        виклики (перевірки, журнал, аудит).
        """
        return student.average_grade
```

```
PS C:\Github\prof_python\Lab_09> student-manager race
8 потоків по 20000 студентів у спільний лічильник групи
без Lock  count=70289 очікувалось=160000 втрачено=89711
з Lock    count=160000 очікувалось=160000 втрачено=0
```

За benchmark без `Lock` у середньому втрачається 48,3 % реєстрацій (5 запусків: 52 626– 86 582 із 160 000), із `Lock` – жодної. Критична секція містить лише читання, обчислення й запис; накладні витрати Lock на типовому інтервалі перемикання (1,30 s проти 1,14 s, різниця 14 %) невеликі порівняно з вартістю втрачених оновлень.

5. Процеси, NumPy і caching

```
def scores_processes(
    grades: Sequence[float], workers: int, chunksize: int | None = None
) -> list[float]:
    """Рейтингові індекси через ProcessPoolExecutor.map.

    chunksize - скільки балів передається воркеру за одне повідомлення
    (pickle). Типово - чотири порції на воркера.
    """
    chunk = chunksize or max(1, len(grades) // (workers * 4))
    with ProcessPoolExecutor(max_workers=workers) as executor:
        return list(executor.map(student_score, grades, chunksize=chunk))
```

```
def scores_worker(
    shard: int,
    grades: Sequence[float],
    results: "Queue[tuple[int, list[float]]]",
) -> None:
    """Виконується в окремому процесі: рахує шматок і кладе в чергу."""
    results.put((shard, scores_chunk(grades)))

def scores_multiprocessing(
    grades: Sequence[float], workers: int
) -> list[float]:
    """Те саме через низькорівневий API: Process на кожен шматок."""
    parts = split(grades, workers)
    results: Queue[tuple[int, list[float]]] = multiprocessing.Queue()
    processes = [
        multiprocessing.Process(
            target=scores_worker, args=(number, part, results)
        )
        for number, part in enumerate(parts)
    ]
    for process in processes:
        process.start()
    collected = dict(results.get() for _ in processes)
    for process in processes:
        process.join()
    return [
        score for number in range(len(parts)) for score in collected[number]
    ]
```

`ProcessPoolExecutor.map` передає воркерам бали порціями `chunksize` (pickle), кожен процес має власний GIL. Додаткові витрати – запуск процесів, серіалізація даних і збір результатів. `multiprocessing.Process` + `Queue` виконує те саме низькорівнево: по процесу на шматок.

```
def to_columns(students: Sequence[Student]) -> StudentColumns:
    """Перетворює список Student на колонки (одноразова вартість)."""
    groups = tuple(sorted({student.group for student in students}))
    codes = {group: number for number, group in enumerate(groups)}
    count = len(students)
    grades = np.fromiter(
        (student.average_grade for student in students),
        dtype=np.float64,
        count=count,
    )
    group_codes = np.fromiter(
        (codes[student.group] for student in students),
        dtype=np.int16,
        count=count,
    )
    return StudentColumns(grades, group_codes, groups)

def scores_numpy(grades: Floats) -> Floats:
    """Векторизована версія student_score для всього масиву одразу."""
    value = grades.copy()
    for _ in range(SCORE_ITERATIONS):
        value = np.sqrt(
            value * value + np.sin(value) ** 2 + np.cos(value) ** 2
        )
    return value
```

NumPy. Python-версія працює зі списком об'єктів, NumPy – з колонками, тому список спершу перетворюється (`to_columns`); цю одноразову вартість показано окремо. **Caching.** Статистика групи кешується через `lru_cache` із ключем (група, версія); додавання студента або зміна балу збільшує версію лише своєї групи.

```
def group_statistics(self, group: str) -> Statistics:
    """Статистика групи; повторний виклик береться з кешу."""
    name = group.strip().upper()
    return self._cached(name, self._versions.get(name, 0))

def _invalidate(self, group: str) -> None:
    """Нова версія групи - новий ключ кешу."""
    self._versions[group] = self._versions.get(group, 0) + 1
```

```
PS C:\Github\prof_python\Lab_09> student-manager cache
Група PI-21: студентів 8199, середній 67.3475
cold cache: 2.870 ms; warm cache: 4.1 µs; cache=CacheStats(hits=2, misses=1, size=1)
після add_student: студентів 8200, середній 67.3515; cache=CacheStats(hits=2, misses=2, size=2)
після change_average_grade: мінімум 0.00, збігається з перерахунком: True; cache=CacheStats(hits=2, misses=3,
size=3)
```

За benchmark (середнє з 5 запусків timeit) cold cache – 6,44 ms, warm cache – 0,36 µs. Для 600 запитів (588 hits, 12 misses) із кешем 0,099 s, без кешу 4,56 s – прискорення 46×. Після додавання студента лічильник групи зріс із 8199 до 8200; після зміни балу кеш віддав значення, що збігається з повним перерахунком.

6. Результати benchmark

Таблиця 2 – Рейтингові індекси (CPU-bound): п'ять запусків, секунди

Dataset	Method	W	Run 1	Run 2	Run 3	Run 4	Run 5	Mean, s	Speedup
10 000	Python	–	0,425	0,412	0,442	0,451	0,447	**0,435**	1,00×
	Threads	8	0,444	0,432	0,460	0,460	0,452	**0,450**	0,97×
	Processes	8	0,292	0,286	0,280	0,286	0,278	**0,284**	1,53×
	multiprocessing	8	1,420	1,408	1,289	1,350	1,435	**1,380**	0,32×
	NumPy	–	0,0407	0,0406	0,0450	0,0427	0,0400	**0,0418**	10,42×
100 000	Python	–	4,379	4,244	5,162	4,130	4,611	**4,505**	1,00×
	Threads	8	4,658	4,180	4,304	4,193	4,629	**4,393**	1,03×
	Processes	8	0,584	0,603	0,599	0,598	0,600	**0,597**	7,55×
	multiprocessing	8	1,533	1,628	1,780	1,588	1,534	**1,613**	2,79×
	NumPy	–	0,527	0,550	0,510	0,578	0,554	**0,544**	8,29×
400 000	Python	–	18,004	17,811	17,886	17,225	17,453	**17,676**	1,00×
	Threads	8	18,306	16,763	18,717	18,013	17,028	**17,765**	0,99×
	Processes	8	1,608	1,603	1,702	1,565	1,985	**1,692**	10,44×
	multiprocessing	8	2,691	2,674	2,787	2,501	3,003	**2,731**	6,47×
	NumPy	–	2,568	2,517	2,509	2,542	2,558	**2,539**	6,96×

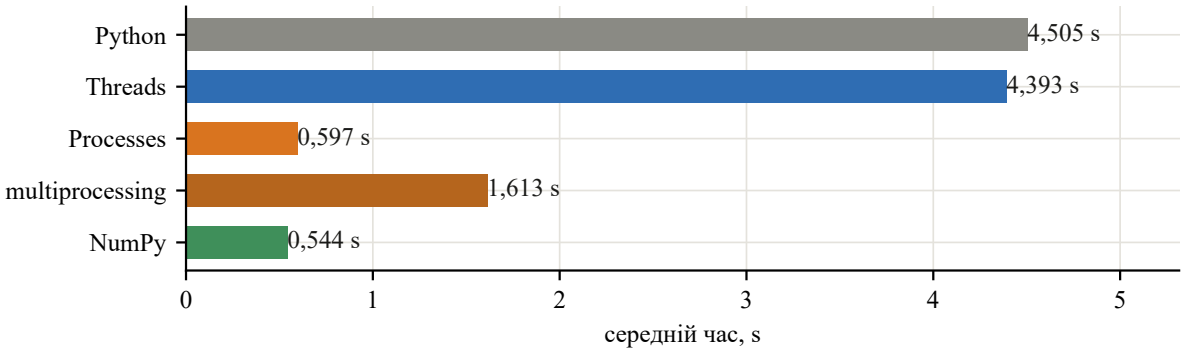


Рисунок 1 – Середній час обчислення індексів, 100 000 студентів (5 повторів)

Таблиця 3 – Інші операції: середній час, s / speedup

Операція	Метод	10 000	100 000	400 000
Статистика (середній, min, max, std)	Python	0,00105 / 1,00×	0,0149 / 1,00×	0,0668 / 1,00×
	Threads	0,00250 / 0,42×	0,0192 / 0,78×	0,0982 / 0,68×
	Threads + Lock	0,00287 / 0,37×	0,0200 / 0,74×	0,104 / 0,64×

Операція	Метод	10 000	100 000	400 000
	Processes	0,254 / 0,00×	0,508 / 0,03×	1,740 / 0,04×
	NumPy	0,00018 / 5,81×	0,00090 / 16,65×	0,00291 / 22,92×
	NumPy + conversion	0,00164 / 0,64×	0,0219 / 0,68×	0,0837 / 0,80×
Статистика за групами	Python	0,00213 / 1,00×	0,0246 / 1,00×	0,123 / 1,00×
	Threads	0,00382 / 0,56×	0,0295 / 0,83×	0,146 / 0,85×
	Processes	0,251 / 0,01×	0,513 / 0,05×	1,773 / 0,07×
	NumPy	0,00097 / 2,21×	0,00369 / 6,67×	0,0140 / 8,82×
	NumPy + conversion	0,00265 / 0,80×	0,0241 / 1,02×	0,0959 / 1,29×
Рейтинг (сортування)	Python	0,00220 / 1,00×	0,0349 / 1,00×	0,169 / 1,00×
	NumPy	0,00078 / 2,81×	0,0126 / 2,76×	0,0520 / 3,24×
	NumPy + conversion	0,00357 / 0,62×	0,0361 / 0,97×	0,135 / 1,25×
Пошук за групою	Python	0,00043 / 1,00×	0,00389 / 1,00×	0,0165 / 1,00×
	NumPy	0,00008 / 5,23×	0,00027 / 14,38×	0,00092 / 17,89×

Таблиця 4 – Кількість workers (100 000 студентів), s / speedup

Workers	Threads, s	Speedup	Processes, s	Speedup
1	4,630	0,94×	2,617	1,66×
2	4,294	1,01×	1,455	2,98×
4	4,414	0,98×	0,879	4,93×
8	4,545	0,95×	0,695	6,24×
16	4,500	0,96×	0,712	6,09×

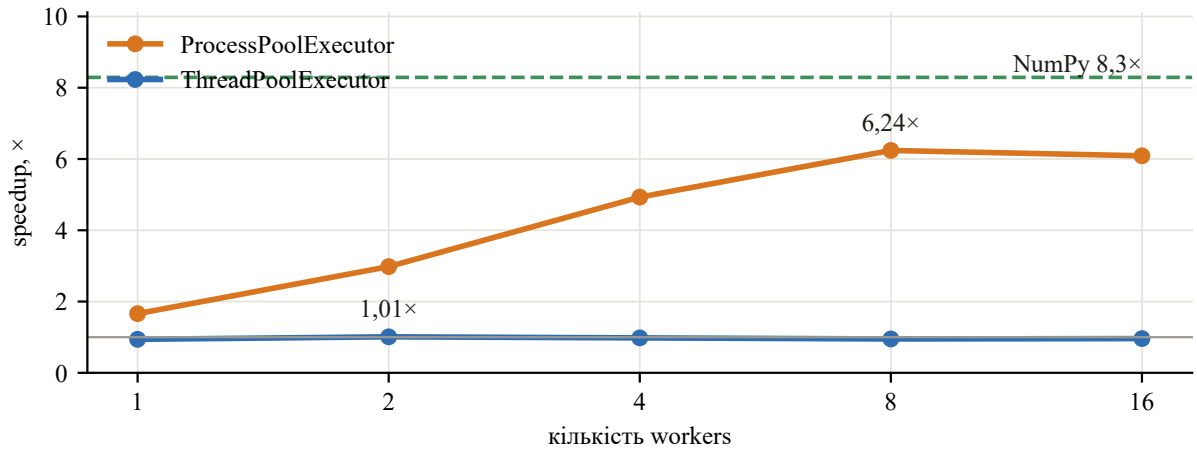


Рисунок 2 – Speedup індексів залежно від кількості workers (100 000 студентів)

Таблиця 5 – Вплив chunksize у ProcessPoolExecutor.map (10 000 студентів, 8 workers)

chunksize	Mean, s	Speedup
chunksize 1	3,534	0,13×
chunksize 10	0,523	0,85×
chunksize 100	0,321	1,38×
chunksize 1000	0,356	1,24×
chunksize 10000	0,464	0,95×

Таблиця 6 – Завантаження 8 файлів (100 000 студентів), I/O-bound

Затримка	Метод	Mean, s	Speedup
0 s (локальний диск)	Sequential	0,521	1,00×
0 s (локальний диск)	Threads	0,500	1,04×
0 s (локальний диск)	Processes	0,842	0,62×
0,1 s на файл	Sequential	1,370	1,00×
0,1 s на файл	Threads	0,630	2,17×
0,1 s на файл	Processes	0,937	1,46×

`to_columns` (list[Student] → масиви NumPy) коштує 0,00193 s на 10 000, 0,0223 s на 100 000 і 0,0871 s на 400 000 студентів.

7. Перевірка правильності й тести

Перед кожним вимірюванням benchmark порівнює результат реалізації з baseline: статистики – з допуском 1e-9, рейтинг – збіг списків індексів (stable sort, рівні бали за порядком), індекси – з допуском 1e-9, файли – рівність списків студентів. Тести додатково порівнюють індекси з аналітичною формулою `sqrt(grade² + 100)`, перевіряють порядок результатів пулів, відсутність втрат під Lock, інвалідацію кешу та I/O (117 tests).

```
PS C:\Github\prof_python\Lab_09> pytest -q --cov=student_manager --cov-branch --cov-report=term-missing
Name                                    Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------------------------
src\student_manager\__init__.py             4      0      0      0   100%
src\student_manager\analytics.py          111      0     26      0   100%
src\student_manager\loaders.py             41      0      6      0   100%
src\student_manager\main.py                91      0     14      2    98%   60->62, 63->exit
src\student_manager\models.py              29      0      8      0   100%
src\student_manager\parallel.py            77      0     10      0   100%
src\student_manager\profiling.py           56      0      2      0   100%
src\student_manager\repositories.py        31      0      0      0   100%
src\student_manager\services.py            61      0      4      0   100%
src\student_manager\shared.py              50      0      6      0   100%
src\student_manager\vectorized.py          43      0      6      0   100%
-----------------------------------------------------------------------------------
TOTAL                                     594      0     82      2    99%
117 passed in 24.21s
```

8. Аналіз експерименту

Питання	Відповідь
Яка функція – bottleneck і яка її частка?	`student_score`: 98,4 % часу звіту за cProfile; решта операцій – соті частки секунди.
CPU-bound чи I/O-bound?	Індекси – CPU-bound (обчислення без очікування). Завантаження файлів – I/O-bound лише за наявності затримки сховища; на локальному диску домінує розбір CSV (CPU).
Чи допоміг ThreadPoolExecutor? Чому?	Для індексів ні: 0,97×–1,03× за будь-якої кількості потоків – один GIL виконує один потік Python bytecode, а керування потоками додає накладні витрати. Для I/O із затримкою так: 2,17× – під час очікування GIL звільняється.

Питання	Відповідь
Чи допоміг ProcessPoolExecutor? Скільки workers оптимально?	Для індексів так: 1,53× на 10 000, 7,55× на 100 000, 10,44× на 400 000 – чим більша робота, тим менша частка запуску процесів. 8 workers дали 6,24×, 16 – 6,09× (виграшу немає), тож оптимум – 8. Для дешевої статистики процеси в 33 разів повільніші: pickle об'єктів дорожчий за саму роботу.
Чи дала перевагу NumPy? Який speedup і як змінилась пам'ять?	Індекси: 8,29× на 100 000 (на 10 000 – 10,42×); статистика: 16,65×, але з перетворенням 0,68×. Колонки займають у 14 разів менше пам'яті, ніж список об'єктів.
Чи дав caching прискорення?	Так: 6,44 ms → 0,36 μs на повторному виклику; потік із 600 запитів – в 46 разів швидше, а після змін кеш віддає свіжі дані.
Чи однаковий результат для малих і великих наборів?	Так, збіг із baseline перевірено на всіх розмірах; змінюється лише speedup (10 000: процеси 1,53×, NumPy 10,42×; 400 000: процеси 10,44×).
Яку реалізацію залишити в production?	Індекси й важкі обчислення – ProcessPoolExecutor із chunksize ≥ 100 для великих наборів (NumPy – для середніх); статистику – NumPy, якщо колонки зберігаються між запитами; завантаження файлів – ThreadPoolExecutor; повторну статистику груп – кеш із версією.

Висновки

Профілювання показало, що bottleneck звіту по студентах – CPU-bound обчислення рейтингового індексу, яке виконує понад 90 % часу. ThreadPoolExecutor через GIL не прискорив його (1,03×), ProcessPoolExecutor дав 10,44× на 400 000 студентів, NumPy – найкращий результат для 10 000–100 000, а дешеві операції (статистика, пошук, рейтинг) виграють від NumPy лише коли дані вже в колонках. Для I/O-bound завантаження файлів корисні потоки, для повторної статистики – caching з invalidation. Отже, оптимальний метод залежить від природи bottleneck і розміру даних, а паралельність без попереднього profiling і вимірювань не гарантує прискорення.